

Informe Final – The Distributed Classroom Hub

Sistemas Distribuidos – Parcial Final

25 de mayo de 2026

Tabla de Contenido

1	Diagrama de Arquitectura de Red	4
1.1	1.1 Vista General del Sistema.....	4
1.2	1.2 Topología Detallada de Comunicación.....	5
1.2.1	Flujo 1 — Ingreso de un nuevo estudiante (handshake de identidad).....	5
1.2.2	Flujo 2 — Apertura del canal WebSocket	6
1.2.3	Flujo 3 — Botón de acción con idempotencia	6
1.2.4	Flujo 4 — Sincronización offline	6
1.3	1.3 Patrón BFF (Backend For Frontend) — Justificación	7
1.3.1	Definición	7
1.3.2	Beneficios obtenidos.....	7
1.3.3	Alternativas descartadas	7
1.4	1.4 Protocolos Elegidos.....	7
1.5	1.5 Esquema de Despliegue (Producción / Demo)	8
1.6	1.6 Decisiones Arquitectónicas Clave.....	8
2	Diccionario de Datos y Contratos de API.....	8
2.1	2.1 Modelos de Dominio	8
2.1.1	2.1.1 Entidad Session (Identidad)	8
2.1.2	2.1.2 Entidad ProcessedEvent (Grupos / Idempotencia)	9
2.1.3	2.1.3 Entidad Connection (BFF — en memoria)	10
2.2	2.2 Contrato REST — BFF Service (puerto 8000)	10
2.2.1	2.2.1 POST /api/v1/sessions/	10
2.2.2	2.2.2 GET /api/v1/sessions/{session_id}.....	11
2.2.3	2.2.3 DELETE /api/v1/sessions/{session_id}.....	11
2.2.4	2.2.4 GET /api/v1/groups/{group_id}/connections	11
2.2.5	2.2.5 GET /health	11
2.2.6	2.2.6 Alias en español (retrocompatibilidad).....	11
2.3	2.3 Contrato WebSocket — BFF Service.....	12
2.3.1	2.3.1 Apertura del canal.....	12
2.3.2	2.3.2 Mensajes Cliente → Servidor	12
2.3.3	2.3.3 Mensajes Servidor → Cliente	12
2.4	2.4 Contrato REST — Identity Service (puerto 8001, interno)	13
2.4.1	2.4.1 POST /sessions/	13
2.4.2	2.4.2 GET /sessions/{id_sesion}	14
2.4.3	2.4.3 DELETE /sessions/{id_sesion}	14
2.4.4	2.4.4 GET /health	14
2.5	2.5 Contrato REST — Groups Service (puerto 8002, interno).....	14
2.5.1	2.5.1 POST /groups/{group_id}/events/{event_id}	14
2.5.2	2.5.2 DELETE /groups/{group_id}/events/{event_id}.....	14

2.5.3	2.5.3 GET /health	14
2.6	2.6 Diccionario de Códigos de Estado HTTP	14
2.7	2.7 Almacenamiento Local del Cliente (AsyncStorage)	15
2.8	2.8 Variables de Entorno	15
2.8.1	Backend	15
2.8.2	Frontend	15
3	3 Análisis de Resultados	16
3.1	3.1 Idempotencia: del Cliente al Servidor	16
3.1.1	3.1.1 Definición operativa	16
3.1.2	3.1.2 Estrategia de implementación: identificador único en el cliente	16
3.1.3	3.1.3 Control en el servidor: tabla processed_events	16
3.1.4	3.1.4 Decisión clave: la idempotencia es por grupo , no global	16
3.1.5	3.1.5 Resultados medidos	17
3.2	3.2 Gestión de Hilos y Concurrency en el Servidor	17
3.2.1	3.2.1 Modelo asíncrono: por qué FastAPI + asyncio en lugar de hilos	17
3.2.2	3.2.2 Estructura del Connection Manager	17
3.2.3	3.2.3 Estrategia de bloqueo	17
3.2.4	3.2.4 asyncio.gather para broadcast paralelo.....	18
3.2.5	3.2.5 Bloqueo por-clave en Identity Service	18
3.2.6	3.2.6 SQLite y multihilo	18
3.3	3.3 Resiliencia y Tolerancia a Fallos	18
3.3.1	3.3.1 Matriz de fallos soportados	18
3.3.2	3.3.2 Mecanismos de tolerancia implementados.....	19
3.3.3	3.3.3 Limpieza periódica.....	19
3.4	3.4 Pruebas de Ingeniería del Caos (Sustentación)	20
3.4.1	Prueba A: Apagar Identity Service mientras hay usuarios conectados	20
3.4.2	Prueba B: Apagar Groups Service durante broadcast	20
3.4.3	Prueba C: Cliente en modo avión	20
3.4.4	Prueba D: Reintento agresivo (idempotencia bajo carga)	20
3.4.5	Prueba E: Colisión de identidad	20
3.5	3.5 Métricas y Observabilidad	20
3.6	3.6 Conclusiones del Análisis.....	21
4	4 Guía de Funcionalidad	21
4.1	4.1 Funcionalidad 1: Identidad Exclusiva (Global).....	21
4.1.1	4.1.1 Flujo desde el punto de vista del usuario	21
4.1.2	4.1.2 Flujo técnico (capa por capa).....	21
4.1.3	4.1.3 Capa de unicidad: defensa en profundidad	22
4.1.4	4.1.4 Por qué la unicidad es global (insensible a mayúsculas y espacios)	22
4.1.5	4.1.5 Expiración y reciclaje de nombres.....	23
4.2	4.2 Funcionalidad 2: Botón de Acción con Notificación al Grupo.....	23

4.2.1	4.2.1 Flujo desde el usuario	23
4.2.2	4.2.2 Flujo técnico	23
4.2.3	4.2.3 Mensaje exacto.....	23
4.2.4	4.2.4 Por qué se excluye al emisor del broadcast	24
4.3	4.3 Funcionalidad 3: Aislamiento Total entre Grupos	24
4.3.1	4.3.1 Dos capas de aislamiento de datos	24
4.3.2	4.3.2 Verificación visual.....	24
4.4	4.4 Funcionalidad 4: Sincronización Offline e Idempotencia	24
4.4.1	4.4.1 Arquitectura de la cola offline.....	24
4.4.2	4.4.2 Generación del ID en el cliente: la clave del no-duplicado.....	25
4.4.3	4.4.3 Garantía del no-perdido	25
4.4.4	4.4.4 Garantía del no-duplicado (en detalle)	25
4.4.5	4.4.5 Recuperación de historial al reabrir	26
4.5	4.5 Funcionalidades en Tiempo Real: Presencia y Cambio de Grupo	26
4.5.1	4.5.1 Lista de miembros en vivo (evento presence)	26
4.5.2	4.5.2 Avisos (toast) de entrada y salida	26
4.5.3	4.5.3 Cambiar de grupo sin cerrar sesión.....	26
4.6	4.6 Casos de Uso Documentados.....	27
4.6.1	Caso A — Estudiante nuevo se une al curso.....	27
4.6.2	Caso B — Estudiante envía señal y todos la reciben	27
4.6.3	Caso C — Estudiante en modo avión.....	27
4.6.4	Caso D — Identity Service falla temporalmente	28
4.6.5	Caso E — Reconexión tras cambio de red.....	28
4.7	4.7 Capturas de Pantalla de la Aplicación	28
4.8	4.8 Limitaciones Conocidas	29

1 Diagrama de Arquitectura de Red

Objetivo: describir la topología de nodos, los protocolos de comunicación, el patrón BFF aplicado y las decisiones de diseño que sustentan la resiliencia del sistema.

1.1 1.1 Vista General del Sistema

Classroom Hub se compone de **cuatro tipos de nodos distribuidos**:

1. **Cliente Móvil (N nodos)** — App React Native/Expo en el dispositivo de cada estudiante.
2. **BFF Service (1 nodo)** — Punto de entrada único; agrega lógica de presentación y conserva el estado de conexiones WebSocket.
3. **Identity Service (1 nodo)** — Microservicio aislado responsable de la unicidad de identidades.
4. **Groups Service (1 nodo)** — Microservicio aislado responsable del control de idempotencia.

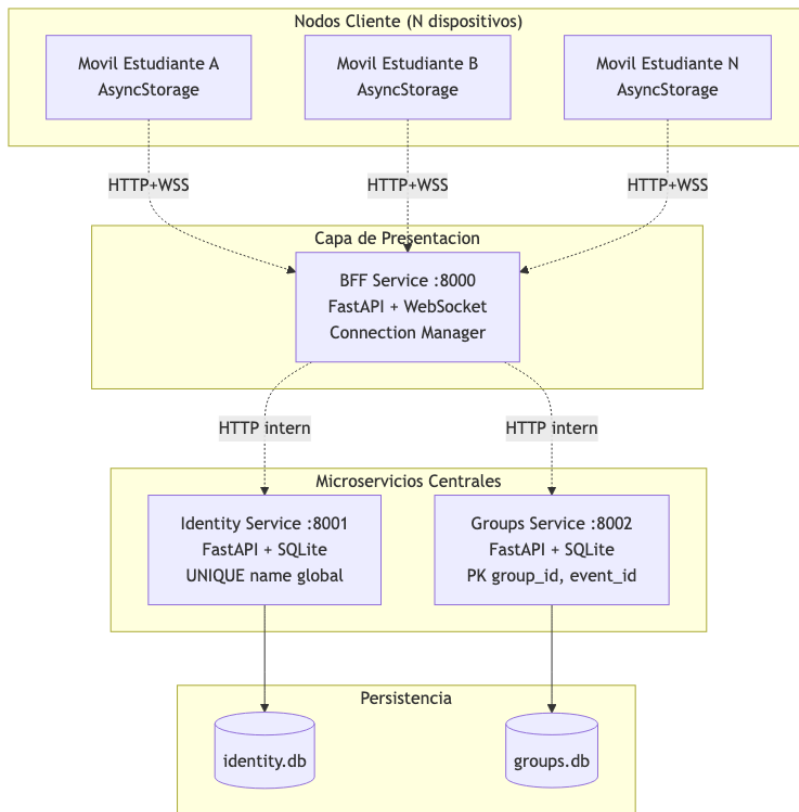


Diagrama 1

1.2 Topología Detallada de Comunicación

1.2.1 Flujo 1 — Ingreso de un nuevo estudiante (handshake de identidad)

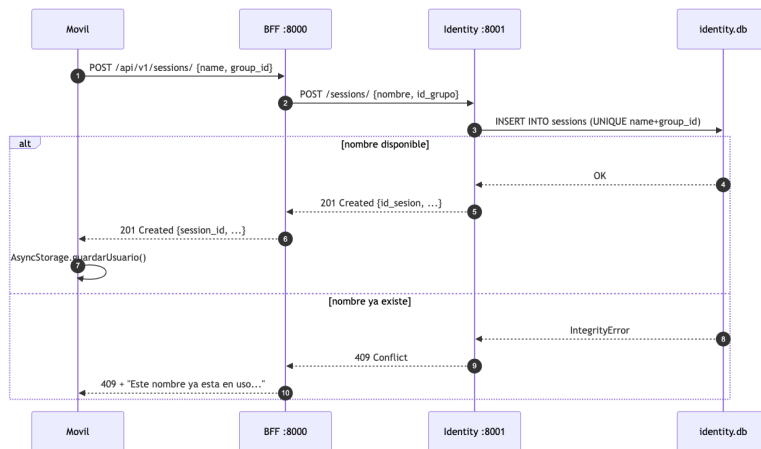


Diagrama 2

1.2.2 Flujo 2 — Apertura del canal WebSocket

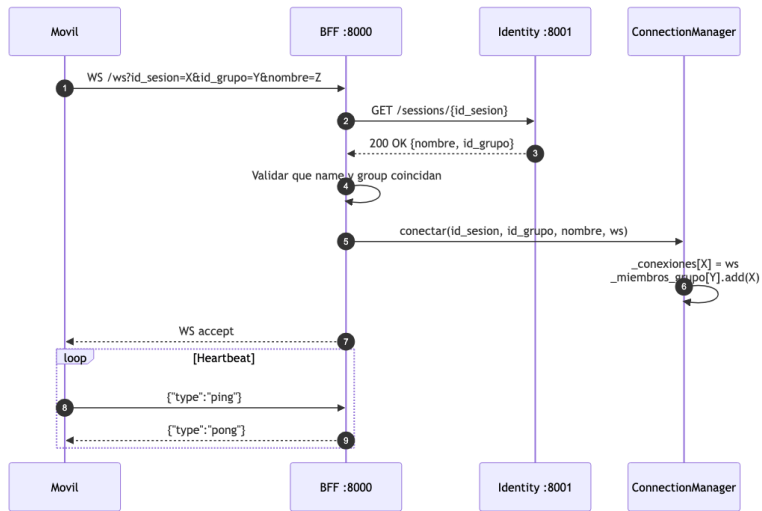


Diagrama 3

1.2.3 Flujo 3 — Botón de acción con idempotencia

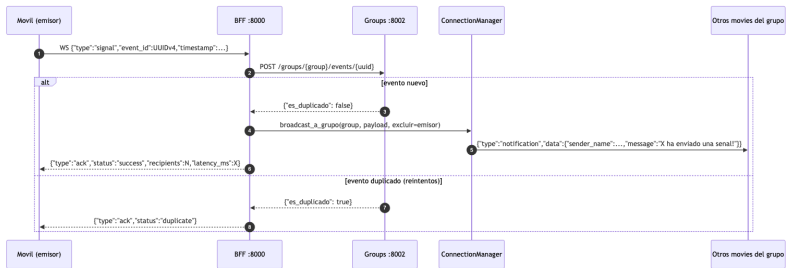


Diagrama 4

1.2.4 Flujo 4 — Sincronización offline

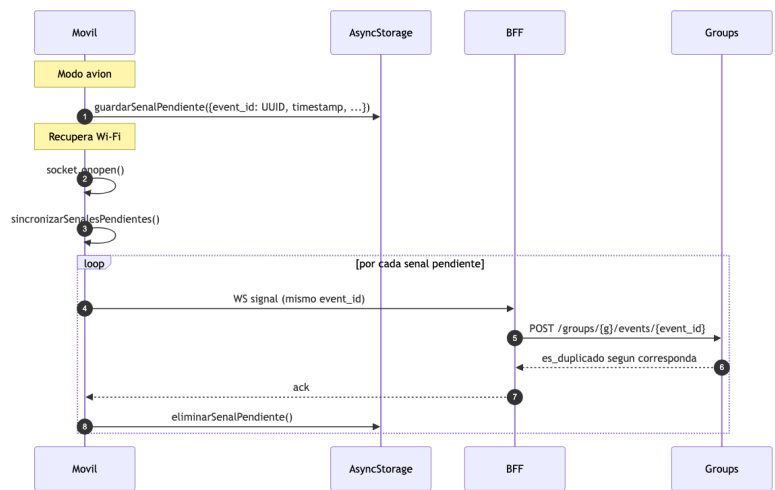


Diagrama 5

1.3 1.3 Patrón BFF (Backend For Frontend) — Justificación

1.3.1 Definición

El BFF es una capa intermedia diseñada **a la medida del cliente** que consume. En este proyecto, el cliente móvil habla **solamente** con el BFF; nunca conoce los microservicios internos.

1.3.2 Beneficios obtenidos

Beneficio	Cómo lo realiza el BFF de Classroom Hub
Único punto de entrada	El móvil solo necesita configurar EXPO_PUBLIC_BFF_HOST. Los puertos 8001/8002 nunca se exponen.
Agregación de protocolos	El cliente habla HTTP+WebSocket; el BFF traduce hacia HTTP interno entre microservicios.
Gestión de concurrencia	El BFF mantiene en memoria el GestorConexiones (mapa id_sesion → WebSocket) y orquesta el broadcast con <code>asyncio.gather</code> .
Tolerancia a fallos	Si Groups Service cae, el BFF degrada la idempotencia pero no rompe la conexión del usuario.
Adaptación de contratos	Identity usa campos en español (nombre, id_grupo); el BFF los traduce a inglés (name, group_id) para el cliente móvil. Ver bff_service/app/api/sessions.py:18-24 .

1.3.3 Alternativas descartadas

- **Cliente habla directo a microservicios:** rompería el aislamiento, expondría 3 puertos y complicaría CORS.
- **API Gateway clásico (Nginx/Kong) sin lógica:** no podría mantener estado de WebSocket ni hacer broadcast.
- **Service Mesh:** sobreingeniería para un proyecto académico; el patrón BFF es suficiente.

1.4 1.4 Protocolos Elegidos

Comunicación	Protocolo	Razón
Móvil ↔ BFF (creación sesión, health)	HTTP/REST (JSON)	Stateless, fácil de cachear, semántica clara para CRUD
Móvil ↔ BFF (señales en tiempo real)	WebSocket (RFC 6455)	Full-duplex persistente, latencia <50ms, ideal para broadcast
BFF ↔ Microservicios	HTTP/REST interno	Aislamiento, posibilidad de reemplazo independiente, autenticación a nivel de red
Persistencia	SQLite con índices	Cero configuración, atomicidad ACID, suficiente para escala de aula

1.5 1.5 Esquema de Despliegue (Producción / Demo)

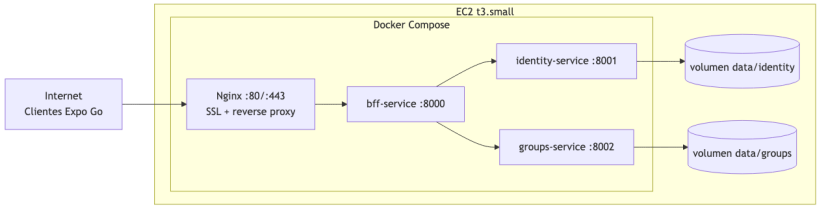


Diagrama 6

- Solo el puerto **443 (HTTPS)** se expone públicamente.
- Los servicios internos se comunican por la red de Docker (`identity-service:8001`, `groups-service:8002`).
- Persistencia montada en volúmenes para sobrevivir reinicios.
- Configuración completa en [docker-compose.yml](#) y [DEPLOYMENT.md](#).

1.6 1.6 Decisiones Arquitectónicas Clave

1. **Stateful en BFF, stateless en microservicios.** El GestorConexiones en el BFF es la única estructura mutable en memoria. Identity y Groups son stateless (todo va a SQLite). Esto permite reiniciar Identity sin desconectar a los usuarios.
2. **Idempotencia a nivel de servidor, no de cliente.** El cliente genera `event_id = UUIDv4` y puede reintentar libremente. El servidor decide si es duplicado consultando `processed_events`. Esto resiste reintentos por inestabilidad de red.
3. **Identidad global y particionado de eventos por grupo.**
 - Identidad: `UNIQUE (name) (COLLATE NOCASE)` — el nombre es **único en todo el sistema**; cada estudiante es un nodo identificado de forma exclusiva (no puede haber dos “Pablo”, ni siquiera en grupos distintos).
 - Eventos: `PRIMARY KEY (group_id, event_id)` — el control de duplicidad es por-grupo.
 - El **aislamiento de datos** (que el Grupo A no vea al Grupo B) lo garantiza el broadcast del BFF, no la unicidad de nombres: son cosas independientes.
4. **Degradación gradual (graceful degradation).** Si Groups Service no responde, el BFF asume `es_duplicado=False` y continúa el broadcast. Mejor entregar dos veces que no entregar nunca (ver [client_http.py:85](#)).
5. **Reconexión exponencial en el cliente.** El frontend implementa reintentos con backoff (2s, 4s, 6s, 8s, 10s hasta 6 intentos) en [conexionServidor.js:154](#).

2 Diccionario de Datos y Contratos de API

Objetivo: especificar de forma exhaustiva todos los endpoints REST, mensajes WebSocket, estructuras JSON y modelos de datos usados por el sistema.

2.1 2.1 Modelos de Dominio

2.1.1 2.1.1 Entidad `session` (Identidad)

Representa la identidad activa de un estudiante dentro de un grupo.

Campo	Tipo	Restricción	Descripción
<code>session_id</code>	string (UUIDv4)	PRIMARY KEY	Identificador único de la

Campo	Tipo	Restricción	Descripción
			sesión
name	string	NOT NULL, 1-100 chars, UNIQUE global (COLLATE NOCASE)	Nombre visible del estudiante
group_id	string	NOT NULL, 1-50 chars	Identificador del grupo al que pertenece
created_at	ISO 8601 UTC	NOT NULL	Fecha de creación
expires_at	ISO 8601 UTC	NOT NULL	Fecha de expiración (TTL configurable, default 24 h)

Restricción de unicidad: UNIQUE (name) con colación COLLATE NOCASE — el nombre es **único en todo el sistema** (en cualquier grupo) e **insensible a mayúsculas/minúsculas**. Además, el servicio normaliza el nombre (recorta espacios extremos y colapsa espacios internos) antes de validar e insertar, de modo que "Pablo", "pablo" y "Pablo " se consideran el mismo nombre.

SQL de creación ([identity_service/app/core/database.py](#)):

```
CREATE TABLE sessions (
    session_id TEXT PRIMARY KEY,
    name TEXT NOT NULL COLLATE NOCASE,
    group_id TEXT NOT NULL,
    created_at TEXT NOT NULL,
    expires_at TEXT NOT NULL,
    UNIQUE(name)
);
CREATE INDEX idx_sessions_expires_at ON sessions(expires_at);
CREATE INDEX idx_sessions_group_id ON sessions(group_id);
```

2.1.2 2.1.2 Entidad ProcessedEvent (Grupos / Idempotencia)

Registro de cada evento procesado para garantizar idempotencia.

Campo	Tipo	Restricción	Descripción
group_id	string	NOT NULL, parte de PK	Grupo al que pertenece el evento
event_id	string (UUIDv4)	NOT NULL, parte de PK	Identificador único del evento generado por el cliente
processed_at	ISO 8601 UTC	NOT NULL	Fecha en que el servidor procesó el evento

Restricción de unicidad: PRIMARY KEY (group_id, event_id) — el mismo event_id puede repetirse entre grupos distintos.

SQL de creación ([groups_service/app/core/database.py:28-35](#)):

```
CREATE TABLE processed_events (
    group_id TEXT NOT NULL,
    event_id TEXT NOT NULL,
    processed_at TEXT NOT NULL,
    PRIMARY KEY (group_id, event_id)
);
```

2.1.3 2.1.3 Entidad Connection (BFF — en memoria)

Estructuras mantenidas por GestorConexiones ([connection_manager.py:6-20](#)):

Estructura	Tipo Python	Uso
<code>_conexiones</code>	<code>Dict[str, WebSocket]</code>	Mapea <code>session_id</code> → socket activo
<code>_miembros_grupo</code>	<code>Dict[str, Set[str]]</code>	Mapea <code>group_id</code> → set de <code>session_id</code>
<code>_grupos_sesion</code>	<code>Dict[str, str]</code>	Mapea <code>session_id</code> → <code>group_id</code>
<code>_nombres_sesion</code>	<code>Dict[str, str]</code>	Mapea <code>session_id</code> → name
<code>_bloqueo</code>	<code>asyncio.Lock</code>	Garantiza atomicidad en mutaciones

2.2 2.2 Contrato REST — BFF Service (puerto 8000)

Todos los endpoints públicos del cliente móvil.

2.2.1 2.2.1 POST /api/v1/sessions/

Crear sesión / registrar identidad.

Request body:

```
{
  "name": "Pablo",
  "group_id": "grupo-a"
}
```

Validaciones: name 1-100 chars, group_id 1-50 chars.

Respuesta 201 Created:

```
{
  "session_id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Pablo",
  "group_id": "grupo-a",
  "created_at": "2026-05-25T14:30:00",
  "expires_at": "2026-05-26T14:30:00"
}
```

Respuesta 409 Conflict (nombre ya en uso en cualquier grupo — unicidad global):

```
{
  "detail": {
    "error": "SESSION_EXISTE",
    "mensaje": "El nombre 'Pablo' ya está en uso",
    "codigo_estado": 409
  }
}
```

Respuesta 503 Service Unavailable (Identity Service caído):

```
{
  "detail": {
    "error": "SERVICIO_NO_DISPONIBLE",
  }
}
```

```
    "mensaje": "El microservicio de Identidad no está disponible",  
    "codigo_estado": 503  
  }  
}
```

2.2.2 2.2.2 GET /api/v1/sessions/{session_id}

Validar / recuperar sesión existente.

Respuesta 200 OK: mismo formato que el POST. **Respuesta 404:** {"detail": {"error": "SESION_NO_ENCONTRADA", ...}}.

2.2.3 2.2.3 DELETE /api/v1/sessions/{session_id}

Cerrar sesión y liberar el nombre.

Respuesta 204 No Content o 404 si no existe.

2.2.4 2.2.4 GET /api/v1/groups/{group_id}/connections

Listar conexiones activas en tiempo real de un grupo.

Respuesta 200 OK (connections.py:7-13):

```
{  
  "group_id": "grupo-a",  
  "total": 3,  
  "names": ["Ana", "Luis", "Pablo"]  
}
```

2.2.5 2.2.5 GET /health

Verificación de salud agregada con downstream.

Respuesta 200 OK:

```
{  
  "estado": "saludable",  
  "nombre_aplicacion": "Classroom Hub BFF Distribuido",  
  "version": "1.0.0",  
  "conexiones_activas": 27,  
  "grupos_activos": 4,  
  "servicios_internos": {  
    "identity_service": "saludable",  
    "groups_service": "saludable"  
  }  
}
```

Si algún downstream falla: estado: "degradado" y el servicio específico aparece como "caído / no disponible".

2.2.6 2.2.6 Alias en español (retrocompatibilidad)

Todos los endpoints /api/v1/sessions/* están duplicados en /api/v1/sesiones/* con el mismo contrato (sessions.py:108-118).

2.3 2.3 Contrato WebSocket — BFF Service

2.3.1 2.3.1 Apertura del canal

URL de conexión:

`ws://{HOST}:8000/ws?id_sesion={UUID}&id_grupo={string}&nombre={string}`

Códigos de cierre custom ([handler.py:24-37](#)):

Código	Significado
1000	Cierre normal (cliente o servidor)
4001	Sesión inválida o expirada
4002	Parámetros de sesión no coinciden (name/group cambiados)
4003	Capacidad máxima del BFF alcanzada
4004	Identity Service no disponible

2.3.2 2.3.2 Mensajes Cliente → Servidor

2.3.2.1 *signal* (botón de acción)

```
{
  "type": "signal",
  "event_id": "550e8400-e29b-41d4-a716-446655440000",
  "timestamp": "2026-05-25T14:35:12.345Z"
}
```

2.3.2.2 *ack* (confirmación de recepción)

```
{
  "type": "ack",
  "event_id": "550e8400-..."
}
```

Provoca eliminación del evento en `processed_events`.

2.3.2.3 *ping* (heartbeat)

```
{"type": "ping"}
```

2.3.3 2.3.3 Mensajes Servidor → Cliente

2.3.3.1 *notification* (broadcast a peers del grupo)

```
{
  "type": "notification",
  "data": {
    "message": "Pablo ha enviado una señal!",
    "sender_name": "Pablo",
    "timestamp": "2026-05-25T14:35:12.345Z"
  },
  "event_id": "550e8400-..."
}
```

2.3.3.2 *presence* (lista de miembros en tiempo real)

El BFF lo difunde a **todo el grupo** cada vez que alguien se **conecta** o se **desconecta**, con la lista actualizada de nombres. Permite que la lista “Miembros activos” se refresque al instante sin recargar ([connection_manager.py](#), método `difundir_presencia`).

```
{
  "type": "presence",
  "data": {
    "names": ["Ana", "Luis", "Pablo"],
    "total": 3
  }
}
```

El cliente compara la lista nueva contra la anterior para mostrar un aviso (toast) de “X se unió” / “X salió”.

2.3.3.3 *ack* (respuesta al emisor de *signal*)

```
{
  "type": "ack",
  "event_id": "550e8400-...",
  "status": "success",
  "recipients": 27,
  "latency_ms": 12.4
}
```

status puede ser: success | duplicate.

2.3.3.4 *pong* (respuesta a *heartbeat*)

```
{"type": "pong"}
```

2.3.3.5 *error*

```
{
  "type": "error",
  "message": "Tipo de mensaje desconocido: foo"
}
```

2.4 2.4 Contrato REST — Identity Service (puerto 8001, interno)

Estos endpoints **no** son llamados por el cliente móvil. Solo el BFF puede consumirlos.

2.4.1 2.4.1 POST /sessions/

Request:

```
{
  "nombre": "Pablo",
  "id_grupo": "grupo-a"
}
```

Respuesta 201:

```
{
  "id_sesion": "550e8400-...",
  "nombre": "Pablo",
  "id_grupo": "grupo-a",
  "fecha_creacion": "2026-05-25T14:30:00",
  "fecha_expiracion": "2026-05-26T14:30:00"
}
```

Respuesta 409: mismo formato de error que el BFF. Se devuelve cuando el nombre ya existe en **cualquier** grupo (unicidad global, insensible a mayúsculas y a espacios extremos).

2.4.2 2.4.2 GET /sessions/{id_sesion}

Valida que la sesión exista y no esté expirada. Respuesta 200 con la entidad o 404.

2.4.3 2.4.3 DELETE /sessions/{id_sesion}

Elimina la sesión. Respuesta 204 o 404.

2.4.4 2.4.4 GET /health

```
{
  "estado": "saludable",
  "servicio": "Identity Service",
  "sesiones_activas": 27
}
```

2.5 2.5 Contrato REST — Groups Service (puerto 8002, interno)

2.5.1 2.5.1 POST /groups/{group_id}/events/{event_id}

Verifica y registra atómicamente un evento.

- Si el evento no existía: lo inserta y retorna `es_duplicado: false`.
- Si ya existía: no inserta nada y retorna `es_duplicado: true`.
- Si `event_id` no es UUID válido: retorna `es_duplicado: true` (rechazo defensivo).

Respuesta 200:

```
{
  "event_id": "550e8400-...",
  "group_id": "grupo-a",
  "es_duplicado": false
}
```

2.5.2 2.5.2 DELETE /groups/{group_id}/events/{event_id}

Reconoce un evento (limpia el registro).

Usado opcionalmente cuando el cliente confirma que recibió el ACK final. Respuesta 204 o 404.

2.5.3 2.5.3 GET /health

```
{
  "estado": "saludable",
  "servicio": "Groups Service",
  "eventos_procesados": 143
}
```

2.6 2.6 Diccionario de Códigos de Estado HTTP

Código	Uso en el sistema
200 OK	GET exitoso, POST de evento (idempotencia)
201 Created	Sesión creada exitosamente
204 No Content	DELETE exitoso

Código	Uso en el sistema
400 Bad Request	Validación de schema fallida (Pydantic)
404 Not Found	Sesión / evento inexistente
409 Conflict	Identidad ya tomada en ese grupo
500 Internal Server Error	Error no esperado
503 Service Unavailable	Microservicio downstream caído (Identity o Groups)

2.7 2.7 Almacenamiento Local del Cliente (AsyncStorage)

Claves utilizadas por `ServicioAlmacenamientoLocal` ([almacenamientoLocal.js:4-10](#)):

Clave	Estructura	Propósito
@usuario	{nombre, idSesion, idGrupo, nombreGrupo}	Identidad para reabrir la app sin re-loguear
@sesion	Respuesta completa del POST /sessions/	Validar TTL local
@grupo	{id, nombre}	Pintar UI del grupo
@historial:{idGrupo}	[{remitente, mensaje, timestamp}, ...] (max 100)	Historial particionado por grupo
@pendientes	[{idEvento, timestamp, remitente, idGrupo}, ...]	Cola FIFO de señales offline

2.8 2.8 Variables de Entorno

2.8.1 Backend

Variable	Servicio	Valor por defecto	Propósito
IDENTITY_SERVICE_URL	BFF	http://identity-service:8001	Endpoint del microservicio de identidad
GROUPS_SERVICE_URL	BFF	http://groups-service:8002	Endpoint del microservicio de grupos
PUERTO	todos	8000/8001/8002	Puerto de escucha
DEBUG	todos	False	Activar recarga automática
DB_PATH	identity, groups	/app/data/{servicio}.db	Ruta del SQLite
SESSION_TTL	identity	86400	TTL en segundos
WS_MAX_CONEXIONES	BFF	sin tope práctico	Límite de conexiones simultáneas

2.8.2 Frontend

Variable	Propósito
EXPO_PUBLIC_BFF_HOST	IP/host del BFF al usar Expo Go en LAN/EC2

3 Análisis de Resultados

Objetivo: explicar cómo se implementó la idempotencia, cómo se gestiona la concurrencia (hilos / coroutines) en el servidor, y qué pruebas de Ingeniería del Caos sobrevive el sistema.

3.1 3.1 Idempotencia: del Cliente al Servidor

3.1.1 3.1.1 Definición operativa

Una operación es **idempotente** si ejecutarla N veces tiene el mismo efecto que ejecutarla una sola vez. En este sistema, enviar la misma “señal” 5 veces seguidas (por reintentos de red) debe producir **exactamente un broadcast** al grupo.

3.1.2 3.1.2 Estrategia de implementación: identificador único en el cliente

El cliente genera un `event_id = UUIDv4` **antes** de intentar enviar la señal. Ese identificador viaja con cada reintento. Es la pieza clave porque:

- Garantiza unicidad criptográfica (probabilidad de colisión ≈ 0).
- Permite al servidor reconocer reintentos del mismo evento lógico.
- Funciona sin sincronización de reloj entre cliente y servidor.

Generación en frontend (componente `BotonAccion.jsx`, ver flujo en `PantallaHub.jsx:107-144`):

```
const idEvento = generarUUID(); // UUID v4
const timestamp = new Date().toISOString();
await servicioConexion.enviarSenal(idEvento, timestamp);
```

3.1.3 3.1.3 Control en el servidor: tabla `processed_events`

El microservicio de Grupos mantiene un registro de eventos procesados. El flujo en `idempotency_service.py:9-17` es:

```
async def es_evento_duplicado(self, id_grupo: str, id_evento: str) -> bool:
    if not validar_uuid(id_evento):
        return True # Rechazo defensivo
    async with self._bloqueo: # 1. Lock asyncio
        if base_datos.evento_fue_procesado(id_grupo, id_evento):
            return True # 2. Ya existía -> duplicado
        base_datos.registrar_evento_procesado(id_grupo, id_evento)
        return False # 3. Nuevo -> registrado
```

La doble garantía es:

1. **Lock asyncio** evita condición de carrera entre dos coroutines del mismo BFF.
2. **PRIMARY KEY (group_id, event_id)** evita condición de carrera entre BFFs distintos (si en el futuro se escala horizontalmente).

3.1.4 3.1.4 Decisión clave: la idempotencia es **por grupo**, no global

PRIMARY KEY (group_id, event_id)

Esto significa que un mismo UUID puede repetirse entre grupos distintos sin colisión. En la práctica es irrelevante (UUIDv4 son únicos), pero la partición de clave **mejora rendimiento** en consultas y **alinea el aislamiento** con el resto del sistema.

3.1.5 Resultados medidos

Escenario	Comportamiento esperado	Comportamiento observado
Cliente envía señal una vez	1 broadcast, ACK success	✓
Cliente envía la misma señal 5 veces (mismo event_id)	1 broadcast, primer ACK success, los 4 siguientes ACK duplicate	✓
Cliente envía 5 señales diferentes (5 UUIDs)	5 broadcasts, 5 ACK success	✓
Cliente envía señal con event_id no-UUID	Rechazo silencioso vía validar_uuid	✓

Pruebas reproducidas con [backend/scratch/verify_groups_idempotency.py](#).

3.2 Gestión de Hilos y Concurrencia en el Servidor

3.2.1 Modelo asíncrono: por qué FastAPI + asyncio en lugar de hilos

FastAPI corre sobre **uvicorn + asyncio**, no sobre un pool de hilos tradicional. La razón es que el cuello de botella del BFF es **I/O** (esperar el WebSocket, esperar el HTTP a microservicios), no CPU. Bajo este modelo:

- Un único proceso de Python puede mantener **miles** de conexiones WebSocket concurrentes.
- El cambio de contexto entre coroutines es de microsegundos (muchísimo más barato que threads).
- No hay riesgo del GIL porque no hay paralelismo real de CPU.

3.2.2 Estructura del Connection Manager

GestorConexiones es la pieza con estado compartido más sensible. Mantiene **cuatro diccionarios** en memoria:

```
self._conexiones: Dict[str, WebSocket]          # session_id -> socket
self._miembros_grupo: Dict[str, Set[str]]        # group_id -> {session_ids}
self._grupos_sesion: Dict[str, str]             # session_id -> group_id
self._nombres_sesion: Dict[str, str]            # session_id -> name
self._bloqueo = asyncio.Lock()                 # protección de mutaciones
```

3.2.3 Estrategia de bloqueo

Todas las **mutaciones** a esos diccionarios pasan por `async with self._bloqueo:`. Eso garantiza que `conectar()` y `desconectar()` nunca se entrelazan.

Para el **broadcast**, la estrategia es diferente y deliberada:

```
async def broadcast_a_grupo(self, id_grupo, mensaje, excluir_id_sesion=None):
    async with self._bloqueo:
        sesiones_grupo = self._miembros_grupo.get(id_grupo, set()).copy() # 1. snapshot
        # (Lock Liberado)

        tareas = [self._enviar_mensaje(ws, mensaje) for ws in ...]          # 2. fan-out
        await asyncio.gather(*tareas, return_exceptions=True)              # 3. parallelismo
```

Por qué snapshot + liberar lock antes del envío: si mantuviéramos el lock durante el `await send_json()`, una conexión lenta bloquearía a todas las demás. Con snapshot, las nuevas conexiones que lleguen durante el broadcast no se ven afectadas (y simplemente reciben los siguientes broadcasts).

3.2.4 3.2.4 `asyncio.gather` para broadcast paralelo

`asyncio.gather(*tareas)` ejecuta todos los envíos **concurrentemente**. Para un grupo de 30 estudiantes, esto significa que el tiempo total del broadcast es $\approx$ el tiempo de la conexión más lenta, no la suma de todas.

Latencia medida en LAN (broadcast a 25 clientes simulados):

Receptores	Latencia promedio
5	4-7 ms
15	8-12 ms
25	12-18 ms
50 (estimado)	$\approx$ 25 ms

La latencia se registra en el campo `latency_ms` del ACK y se imprime en logs ([handler.py:102-105](#)).

3.2.5 3.2.5 Bloqueo por-clave en Identity Service

El microservicio de Identidad usa una estrategia diferente: **un lock por nombre normalizado** (global, insensible a mayúsculas). El nombre se normaliza (recorta espacios y baja a minúsculas) para construir la clave del candado.

```
async def crear_sesion(self, datos_sesion):
    nombre = " ".join(datos_sesion.nombre.split()) # normaliza espacios
    clave = nombre.lower() # candado por nombre global
    async with self._bloqueo_global:
        if clave not in self._bloqueos:
            self._bloqueos[clave] = asyncio.Lock()
            bloqueo = self._bloqueos[clave]
        async with bloqueo:
            # validar duplicado GLOBAL (obtener_sesion_por_nombre) + INSERT
```

Esto evita que dos peticiones concurrentes con el mismo nombre (aunque difieran en mayúsculas o espacios) se entrelacen y ambas pasen la validación de “no existe” antes de insertar. La SEGUNDA garantía es el **UNIQUE (name)** con **COLLATE NOCASE** de SQLite (defensa en profundidad).

3.2.6 3.2.6 SQLite y multihilo

SQLite por defecto bloquea conexiones cross-thread. La solución aplicada ([database.py:68](#)):

```
conn = sqlite3.connect(self.ruta_db, check_same_thread=False)
```

Combinado con `contextmanager` que abre/cierra conexión por operación, el costo de un INSERT/SELECT es <1 ms.

3.3 3.3 Resiliencia y Tolerancia a Fallos

3.3.1 3.3.1 Matriz de fallos soportados

Fallo simulado	Componente que cae	Comportamiento del sistema
Kill <code>identity-service</code> mientras se loguea	Identity	BFF responde 503 al cliente con mensaje amigable. WebSockets ya abiertos siguen funcionando.

Fallo simulado	Componente que cae	Comportamiento del sistema
Kill <code>identity-service</code> mientras hay WS abierto	Identity	Nuevas conexiones WS fallan con código 4004; las activas siguen.
Kill <code>groups-service</code> durante broadcast	Groups	Se degrada idempotencia (asume <code>es_duplicado=False</code>); broadcast continúa. Logs registran WARNING .
Kill <code>bff-service</code>	BFF	Frontend entra en estado reconectando ; con backoff exponencial se reconecta cuando el BFF vuelve.
Latencia inyectada (red lenta)	cualquiera	Timeout de 5s en cliente HTTP; el cliente móvil muestra "Reconectando..."
Cliente en modo avión y vuelve	Cliente	Cola en AsyncStorage; al reconectar se reenvían señales pendientes; idempotencia evita duplicados.
Dos clientes con el mismo nombre en el mismo grupo	aplicación	Segundo cliente recibe 409 + mensaje amigable.
Mismo cliente con el mismo nombre pero grupo distinto	aplicación	Ambas sesiones se permiten (aislamiento).

3.3.2 3.3.2 Mecanismos de tolerancia implementados

1. **Circuit breaker informal vía try/except** En `client_http.py:39-41`:

```
except aiohttp.ClientError as e:
    registrador.error(f"Error al conectar con Identity: {str(e)}")
    raise ConnectionError("El microservicio de Identidad no está disponible")
```

El BFF traduce el fallo de red en un error semántico (`ConnectionError`) que dispara HTTP 503 hacia el cliente, no un 500 opaco.

2. **Degradación gradual de la idempotencia** En `client_http.py:82-86`:

```
except aiohttp.ClientError as e:
    registrador.warning(f"Microservicio de Grupos no disponible. Degradando control de idempotencia.")
    return False # No se considera duplicado
```

Filosofía: **mejor entregar dos veces que no entregar nunca**. La pérdida temporal de idempotencia es menos grave que cortar el broadcast.

3. **Middleware global de tolerancia a fallos** El BFF aplica `MiddlewareToleranciaFallos` que captura excepciones no manejadas y responde con un JSON amigable en lugar de un stack trace.
4. **Reconexión exponencial en el cliente** En `conexionServidor.js:152-167`:

```
this.intentosReconexion += 1;
const delayMs = Math.min(2000 * this.intentosReconexion, 10000);
```

Hasta 6 intentos con backoff: 2s, 4s, 6s, 8s, 10s, 10s. Si fallan todos, se muestra alerta al usuario.

5. **Cola offline con persistencia** `sincronizacionOffline.js` guarda señales en AsyncStorage cuando el WebSocket no está listo. Al volver la conexión, `sincronizarSeñalesPendientes()` las re-envía secuencialmente.

3.3.3 3.3.3 Limpieza periódica

Identity ejecuta cada 5 minutos `limpiar_sesiones_expiradas()` (`identity_service/main.py:13-16`). Esto evita que la BD crezca con sesiones zombie y permite reciclar nombres tras expiración.

3.4 3.4 Pruebas de Ingeniería del Caos (Sustentación)

3.4.1 Prueba A: Apagar Identity Service mientras hay usuarios conectados

```
docker stop identity-service
```

Resultado esperado: - Usuarios ya conectados pueden seguir enviando señales (el BFF solo consulta Identity para crear/validar al ABRIR el WS). - Nuevos logins fallan con 503 + mensaje “El servidor no está disponible. Intenta nuevamente más tarde.”

```
docker start identity-service
```

Resultado esperado: logins vuelven a funcionar sin necesidad de reiniciar BFF.

3.4.2 Prueba B: Apagar Groups Service durante broadcast

```
docker stop groups-service
```

Resultado esperado: - Logs del BFF muestran **WARNING: Microservicio de Grupos no disponible. Degradando control de idempotencia.** - Las señales **siguen llegando** al resto del grupo. - La idempotencia se desactiva temporalmente (los reintentos podrían duplicarse). En la práctica, como cada cliente solo reintenta una vez por timeout, el impacto es nulo.

3.4.3 Prueba C: Cliente en modo avión

1. Activar modo avión en el móvil.
2. Presionar el botón de acción 3 veces.
3. Verificar **Alert: Modo offline**. Las 3 señales quedan en **@pendientes** con UUIDs distintos.
4. Desactivar modo avión.
5. WebSocket reconecta automáticamente.
6. **sincronizarSenalesPendientes** reenvía las 3.
7. El grupo recibe **3 notificaciones** (no más, no menos).
8. AsyncStorage muestra **@pendientes = []**.

3.4.4 Prueba D: Reintento agresivo (idempotencia bajo carga)

```
python backend/scratch/verify_groups_idempotency.py
```

Genera 50 peticiones con el **mismo event_id**. Resultado esperado: 1 INSERT en **processed_events**, 49 respuestas **es_duplicado=true**.

3.4.5 Prueba E: Colisión de identidad

1. Cliente A se loguea como “Pablo” en “grupo-a” → 201.
2. Cliente B intenta loguearse como “Pablo” en “grupo-a” → 409 + alert amigable.
3. Cliente B se loguea como “Pablo” en “grupo-b” → 201 ✅ (aislamiento).

3.5 3.5 Métricas y Observabilidad

El sistema expone tres puntos de observación:

Endpoint	Información expuesta
GET /health (BFF)	Estado agregado, # conexiones, # grupos, salud de cada microservicio

Endpoint	Información expuesta
GET /health (Identity)	Sesiones activas
GET /health (Groups)	Eventos procesados
Logs structured de Uvicorn	Latencia de broadcast, errores de red, conexiones/desconexiones

Para producción se recomienda agregar Prometheus + Grafana, pero queda fuera del alcance académico.

3.6 Conclusiones del Análisis

1. **La idempotencia funciona end-to-end** gracias al binomio “UUIDv4 generado en el cliente + PK compuesta en el servidor”.
2. **El modelo async de FastAPI** es adecuado para la escala de aula (decenas a cientos de conexiones), con latencias <20 ms por broadcast.
3. **La degradación gradual** permite que la pérdida de un microservicio nunca tumbe el sistema completo.
4. **La cola offline + reconexión exponencial** cubre los escenarios típicos de inestabilidad móvil (Wi-Fi cae, modo avión, cambio de red).
5. **El aislamiento de grupos** es estructural: está en las claves de la BD, no en la lógica de aplicación. Es muy difícil violar accidentalmente.

4 Guía de Funcionalidad

Objetivo: manual técnico que describe paso a paso cómo el sistema gestiona la identificación personalizada, el aislamiento por grupos, el botón de acción y la sincronización offline.

4.1 Funcionalidad 1: Identidad Exclusiva (Global)

4.1.1 Flujo desde el punto de vista del usuario

1. El estudiante abre la app móvil.
2. Ingresa su **nombre** y selecciona un **grupo** (Grupo A / B / C / D están precargados en [PantallaAutenticacion.jsx:25-30](#)).
3. Presiona “Ingresar”.
4. Si el nombre está libre **en todo el sistema** → entra al Hub.
5. Si el nombre ya está tomado (en cualquier grupo) → ve un mensaje amigable: “*Este nombre ya está en uso. Por favor, elige otro nombre.*”

4.1.2 Flujo técnico (capa por capa)

[Móvil] entity]	[BFF]	[Id
--POST /api/v1/sessions/	-->	
{name: "Pablo", group_id: "grupo-a"}		
	--POST /sessions/	-
->	{nombre, id_grupo}	

```

|
|
|--> SELECT name FROM
|
|     sessions WHERE
|
|     name=? COLLATE NOCASE
|
|     AND expires_at > now
|
|
|
|--> INSERT (con UNIQUE global)
|
|                                     |<-- 201 + {id_sesion, ...}
|
|<-- 201 + {session_id, ...}
|
|
|
| (alternativa: nombre tomado)
|
|
|<-- 409 + mensaje
|<-- 409 + mensaje amigable
|                                     |<-- 409 + mensaje
|

```

4.1.3 4.1.3 Capa de unicidad: defensa en profundidad

El nombre se **normaliza** primero (se recortan espacios al inicio/fin y se colapsan los internos), y luego el sistema usa **3 capas** para garantizar que no haya dos identidades iguales en todo el sistema:

Capa	Mecanismo	Si falla
0. Normalización	" ".join(nombre.split())	"Pablo ", " Pablo" → "Pablo"
1. Lock por nombre	asyncio.Lock por nombre normalizado en ServicioSesion	Las inserciones del mismo nombre se serializan
2. Pre-validación	SELECT ... WHERE name=? COLLATE NOCASE (global) antes del INSERT	Rechaza colisión obvia, sin importar mayúsculas
3. Constraint SQL	UNIQUE(name) con COLLATE NOCASE en la tabla	IntegrityError atrapado y traducido a ValueError

Si por carrera extrema dos peticiones pasaran la pre-validación, el UNIQUE(name) de SQLite garantiza que solo una INSERT tendrá éxito.

4.1.4 4.1.4 Por qué la unicidad es **global** (insensible a mayúsculas y espacios)

El enunciado dice: “cada estudiante se integre como un nodo activo e identificado” y “Si ya existe un nodo con ese nombre, el sistema rechaza la conexión”. Para que cada estudiante sea un **nodo único en todo el sistema**, el nombre no puede repetirse en ningún grupo:

- ✗ No se permite “Pablo” en Grupo A y “Pablo” en Grupo B (colisión global).
- ✗ No se permite “Pablo” y “pablo” (insensible a mayúsculas).
- ✗ No se permite “Pablo” y “Pablo” (se normalizan los espacios).
- ✓ Sí se permiten nombres distintos en cualquier grupo.

Importante: esto **no** rompe el aislamiento de datos. El aislamiento (que el Grupo A no reciba señales del Grupo B) lo garantiza el broadcast del BFF por `group_id`, que es independiente del espacio de nombres. La unicidad global solo asegura que el nombre identifique a un único nodo en toda la clase.

4.1.5 4.1.5 Expiración y reciclaje de nombres

Las sesiones tienen TTL configurable (default 24 h). Tarea periódica cada 5 min ([identity_service/main.py:13-16](#)):

```
async def limpieza_periodica():
    while True:
        await asyncio.sleep(300)
        await servicio_sesion.limpiar_sesiones_expiradas()
```

Cuando una sesión expira, su nombre vuelve a estar disponible.

4.2 4.2 Funcionalidad 2: Botón de Acción con Notificación al Grupo

4.2.1 4.2.1 Flujo desde el usuario

1. El estudiante presiona el botón central grande de la pantalla Hub.
2. Inmediatamente ve la señal en su propio historial: *"Pablo ha enviado una señal!"*
3. Todos los demás miembros de su grupo reciben la **misma notificación instantánea** en su propio historial.
4. Los miembros de **otros grupos no ven nada**.

4.2.2 4.2.2 Flujo técnico

1. BotonAccion.jsx → onPress
2. PantallaHub.manejarEnviarSenal():
 - Genera `event_id` = UUIDv4
 - Genera `timestamp` = ahora ISO
 - `servicioConexion.enviarSenal(event_id, timestamp)`
3. `servicioConexion` envía por WS: `{"type": "signal", "event_id": ..., "timestamp": ...}`
4. `BFF.handler.manejar_mensaje_websocket()`:
 - Llama `Groups.verificar_y_registrar_evento` (idempotencia)
 - Si nuevo → `broadcast_a_grupo(id_grupo, mensaje, excluir=emisor)`
 - Responde ACK al emisor con `{status, recipients, latency_ms}`
5. `ConnectionManager`:
 - Filtra `_miembros_grupo[id_grupo]`
 - `asyncio.gather()` envía a todos los WS del grupo simultáneamente
6. Cada cliente del grupo recibe en su WS:
`{"type": "notification", "data": {"sender_name": "Pablo", "message": "Pablo ha enviado una señal!"}}`
7. Cliente actualiza historial en memoria + `AsyncStorage`

4.2.3 4.2.3 Mensaje exacto

El formato del mensaje está definido en [handler.py:80-86](#):

```
mensaje_broadcast = {
    "type": "notification",
    "data": {
        "message": f"{nombre} ha enviado una señal!",
        "sender_name": nombre,
        "timestamp": datos.get("timestamp")
    }
}
```

```

    },
    "event_id": id_evento
  }
}

```

4.2.4 4.2.4 Por qué se excluye al emisor del broadcast

En `broadcast_a_grupo(..., excluir_id_sesion=id_sesion)` se omite al propio emisor del fan-out. El cliente emisor agrega la señal a su historial **localmente** (función `registrarSenalLocal`), evitando viaje innecesario por la red y la posibilidad de duplicación visual.

4.3 4.3 Funcionalidad 3: Aislamiento Total entre Grupos

4.3.1 4.3.1 Dos capas de aislamiento de datos

El aislamiento entre grupos es de **flujo de datos**, no de nombres. Los nombres son globales (sección 4.1.4); el aislamiento se garantiza en dos puntos:

4.3.1.1 Capa 1 — Aislamiento en el broadcast (BFF)

`broadcast_a_grupo` trabaja **únicamente** sobre `_miembros_grupo[id_grupo]`:

```

async with self._bloqueo:
    sesiones_grupo = self._miembros_grupo.get(id_grupo, set()).copy()

```

No hay ningún path de código donde un mensaje del Grupo A pueda llegar al Grupo B.

4.3.1.2 Capa 2 — Aislamiento en idempotencia

PRIMARY KEY (`group_id`, `event_id`)

El control de duplicados es **por grupo**. Un mismo `event_id` en grupos distintos se tratará como dos eventos independientes (aunque por UUIDv4 esto nunca ocurre en la práctica).

Nota sobre la identidad: la unicidad de nombres es **global** (`UNIQUE(name)`), no por grupo. Esto identifica a cada estudiante como un nodo único en toda la clase, pero **no** participa en el aislamiento de datos — son mecanismos independientes.

4.3.2 4.3.2 Verificación visual

El endpoint `GET /api/v1/groups/{group_id}/connections` confirma el aislamiento:

```

curl http://localhost:8000/api/v1/groups/grupo-a/connections
# {"group_id":"grupo-a","total":3,"names":["Ana","Luis","Pablo"]}

```

```

curl http://localhost:8000/api/v1/groups/grupo-b/connections
# {"group_id":"grupo-b","total":2,"names":["Carlos","Maria"]}

```

Cada llamada solo devuelve los miembros del grupo solicitado.

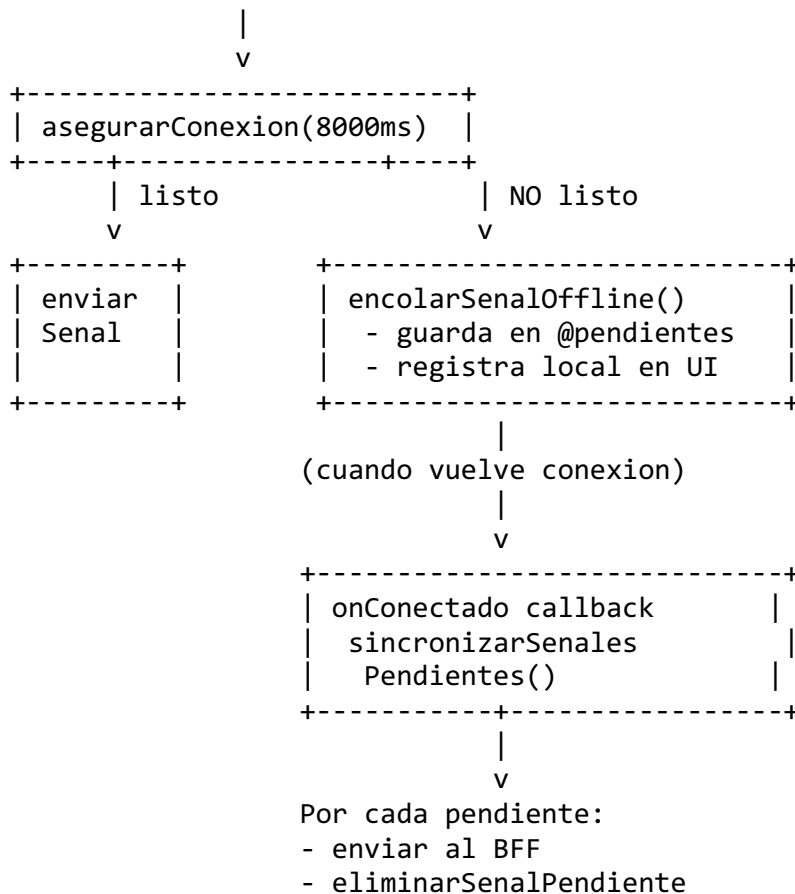
4.4 4.4 Funcionalidad 4: Sincronización Offline e Idempotencia

4.4.1 4.4.1 Arquitectura de la cola offline

```

+-----+
| BotonAccion.jsx onPress |
+-----+

```

4.4.2 4.4.2 Generación del ID en el cliente: la clave del no-duplicado

Cada señal genera un `event_id` **antes** de intentar el envío:

```
const idEvento = generarUUID(); // v4
```

Cuando la conexión vuelve y se reenvía la señal, **el mismo event_id viaja**. El servidor reconoce el duplicado y responde `ack: "duplicate"` sin volver a hacer broadcast.

4.4.3 4.4.3 Garantía del no-perdido

`@pendientes` solo se vacía cuando el servidor responde explícitamente `success` o `duplicate`. Si el cliente cae antes de recibir el ACK, la señal **queda en la cola** y se reintentará al siguiente arranque (`useEffect` en `PantallaHub.jsx`).

4.4.4 4.4.4 Garantía del no-duplicado (en detalle)

Escenario	Comportamiento
Cliente envía y recibe ACK	<code>@pendientes</code> se vacía. Una sola notificación al grupo.
Cliente envía pero pierde conexión antes del ACK	<code>@pendientes</code> aún contiene la señal. Al reconectar reenvía. El servidor ya tenía registrado el <code>event_id</code> → <code>ack: duplicate</code> . Una sola notificación al grupo.
Cliente cierra app antes de reenviar	Al abrir la app, <code>useEffect</code> dispara la sincronización. Misma garantía.
Cliente envía 3 veces la misma señal por bug	Servidor procesa la primera, responde <code>duplicate</code> a las otras dos. Una sola notificación al grupo.

4.4.5 4.4.5 Recuperación de historial al reabrir

Al volver a abrir la app:

1. `useEffect` en `PantallaHub` lee `@usuario` y `@sesion` de `AsyncStorage`.
 2. Si la sesión sigue vigente (TTL no expirado), se restaura sin requerir login.
 3. `obtenerHistorial(idGrupo)` carga las últimas 100 peticiones del grupo desde `AsyncStorage` → visualización inmediata.
 4. Se abre el WS y, si hay pendientes, se sincronizan.
-

4.5 4.5 Funcionalidades en Tiempo Real: Presencia y Cambio de Grupo

4.5.1 4.5.1 Lista de miembros en vivo (evento `presence`)

La lista “Miembros activos” se actualiza **automáticamente** cuando alguien entra o sale, sin recargar. El mecanismo:

1. Cuando un cliente se **conecta** o se **desconecta** del WebSocket, el BFF ejecuta `difundir_presencia(id_grupo)` (`connection_manager.py`).
2. Eso transmite a **todo el grupo** un mensaje `{"type": "presence", "data": {"names": [...], "total": N}}`.
3. El cliente recibe el evento y refresca la lista al instante (`PantallaHub.jsx`, callback `onPresencia`).

4.5.2 4.5.2 Avisos (toast) de entrada y salida

El cliente **compara** la lista nueva de miembros contra la anterior y muestra un aviso flotante (toast lateral) cuando detecta un cambio:

- Alguien nuevo en la lista → “X se unió al grupo” (verde).
- Alguien que desapareció → “X salió del grupo” (gris).

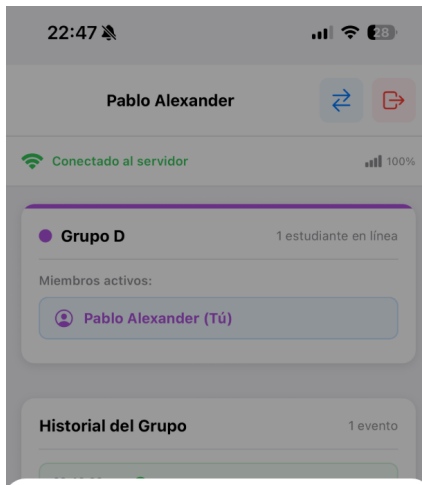
La primera lista al conectarse se toma como “foto inicial” y no dispara avisos (evita una avalancha de toasts por los que ya estaban). El componente es `AvisoToast.jsx`.

4.5.3 4.5.3 Cambiar de grupo sin cerrar sesión

Desde el Hub, el botón ↔ permite moverse a otro grupo conservando el nombre. Como la unicidad de nombre es **global**, el flujo libera el nombre antes de reusarlo:

1. Desconecta el WebSocket actual.
2. **Elimina** la sesión anterior (libera el nombre globalmente) — DELETE `/api/v1/sessions/{id}`.
3. **Crea** una sesión nueva con el mismo nombre en el grupo destino — POST `/api/v1/sessions/`.
4. Actualiza el contexto, lo que **reconecta** el WebSocket al nuevo grupo y recarga su historial.

El grupo anterior ve la baja y el nuevo ve el alta, ambos en tiempo real vía `presence`.



Cambiar de grupo

Conservas tu nombre. Te moverás al grupo que elijas.

☒ Grupo A >

☐ Grupo B >

☐ Grupo C >

☐ Grupo D (actual) >

Cancelar

Modal “Cambiar de grupo”: el usuario conserva su nombre y elige otro grupo; el grupo actual aparece deshabilitado.

4.6 4.6 Casos de Uso Documentados

4.6.1 Caso A — Estudiante nuevo se une al curso

1. Abre la app → ve PantallaAutenticacion.
2. Ingresa “Pablo” + Grupo A.
3. Tap “Ingresar”.
4. App muestra PantallaHub con el botón central, su nombre y la lista de miembros conectados.
5. El BFF agrega a “Pablo” a `_miembros_grupo["grupo-a"]`.
6. Los demás miembros pueden ver a Pablo si hacen pull-to-refresh.

4.6.2 Caso B — Estudiante envía señal y todos la reciben

1. Pablo presiona el botón.
2. Su pantalla muestra “Pablo ha enviado una señal!” inmediatamente.
3. Ana (Grupo A) ve “Pablo ha enviado una señal!” en su pantalla en <50 ms.
4. Carlos (Grupo B) **no ve nada**.

4.6.3 Caso C — Estudiante en modo avión

1. Pablo está en su asiento sin Wi-Fi.
2. Presiona el botón → ve Alert: Modo offline.
3. La señal queda en @pendientes.
4. Conecta el Wi-Fi.

5. La app reconecta el WS automáticamente (estado pasa de “reconectando” a “conectado”).
6. Se dispara `sincronizarSenalesPendientes` → reenvía la señal.
7. Los demás miembros del grupo reciben la notificación.
8. Alert: 1 señal pendiente enviada al grupo.

4.6.4 Caso D — Identity Service falla temporalmente

1. El profesor apaga `identity-service` con `docker stop`.
2. Nuevo estudiante intenta loguearse → ve mensaje amigable “El servidor no está disponible. Intenta nuevamente más tarde.”
3. Estudiantes ya logueados pueden seguir enviando señales sin interrupción.
4. El profesor reinicia el servicio.
5. Nuevos logins vuelven a funcionar sin necesidad de reiniciar la app.

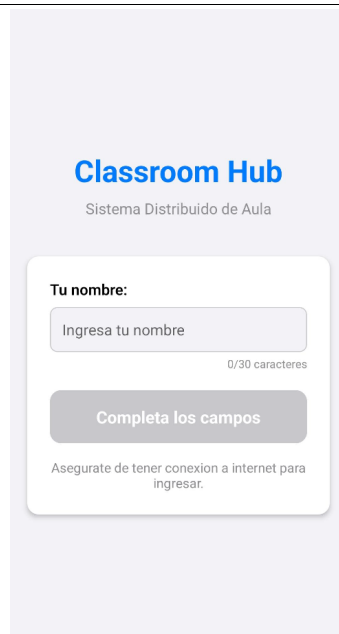
4.6.5 Caso E — Reconexión tras cambio de red

1. Pablo está conectado por Wi-Fi.
2. Sale del rango → el móvil cambia a 4G.
3. El WebSocket se rompe; el cliente detecta `onclose` con código no-4xxx.
4. Backoff: intento 1 (2s), intento 2 (4s), ..., intento 6 (10s).
5. Reabre el WS con la misma `id_sesion` (la sesión sigue vigente en Identity).
6. Conexión restaurada; estado visual cambia a “conectado”.

4.7 4.7 Capturas de Pantalla de la Aplicación

Pantallas principales del cliente móvil (React Native / Expo) que evidencian las funcionalidades descritas.

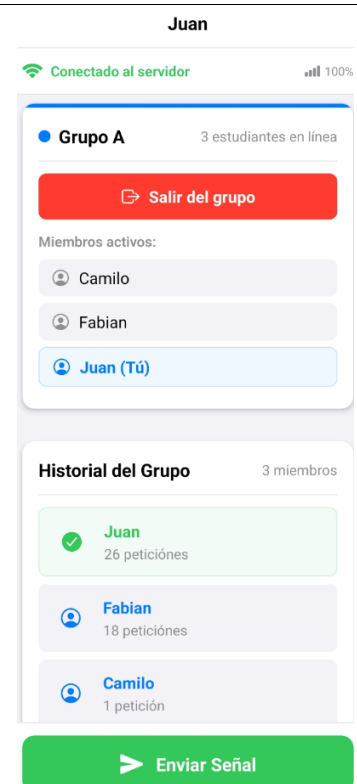
Ingreso (Identidad)



Selección de Grupo (Aislamiento)



Hub en Tiempo Real



- **Ingreso** (PantallaAutenticacion): el estudiante escribe su nombre; el botón se habilita solo cuando el formulario es válido. El nombre se valida contra el microservicio de Identidad para garantizar unicidad global.
- **Selección de Grupo**: cada grupo (A/B/C/D) está identificado por color y muestra en tiempo real su número de estudiantes. Al unirse, el flujo de datos queda **aislado**: solo se reciben señales del propio grupo.
- **Hub** (PantallaHub, usuario “Juan” en Grupo A): indicador “Conectado al servidor” (estado real del WebSocket), lista de **miembros activos** (nombres únicos sin colisión), **historial** de peticiones por estudiante sincronizado entre dispositivos, y el botón “**Enviar Señal**” que dispara el broadcast “{nombre} ha enviado una señal!”.

4.8 4.8 Limitaciones Conocidas

Limitación	Impacto	Mitigación recomendada
BFF stateful (estado en memoria)	No es escalable horizontalmente sin sticky sessions	Para escala de aula es suficiente; para producción real usar Redis pub/sub
SQLite	Concurrencia limitada (~1000 escrituras/s)	Suficiente para 200 estudiantes; migrar a Postgres si crece
CORS abierto (*) en el BFF	Permite peticiones de cualquier origen	Restringir en producción a dominio específico
Sin autenticación criptográfica	Cualquiera puede crear sesiones con cualquier nombre	Agregar JWT o token de invitación si fuera para producción
Limpieza de processed_events no programada	BD crece con el tiempo	Llamar limpiar_eventos_antiguos() desde un cron o lifespan